

Engineered Features & Model Selection Summary

Date: 2026-08-05

This two-page summary covers: engineered feature list, cross-validation (CV) comparison plots/tables, statistical test results, and recommended models/features to tune.

****Engineered Feature List****

- age_bucket — categorical bins: ≤ 25 , 26-35, 36-45, 46-55, 56-65, 65+. Justification: captures non-linear age-income relationship.
- hours_bucket — categorical bins: part_time_le20, reduced_21_35, standard_36_40, over_41_50, heavy_50plus. Justification: distinguishes part-time vs overtime pay regimes.
- has_capital_gain — binary flag (capital-gain > 0). Justification: threshold signal for investment income.
- has_capital_loss — binary flag (capital-loss > 0). Justification: signals loss/investment activity.
- higher_education — binary flag (education-num ≥ 13). Justification: interpretable cutoff for degree effect.
- log_capital_gain — numeric transform: $\log_{1p}(\text{capital-gain})$. Justification: compresses heavy right tail for continuous models.
- edu_hours_interaction — numeric interaction: education-num * hours-per-week. Justification: captures compounding effect of education and hours.
- net_capital — numeric: capital-gain - capital-loss. Justification: single investment-activity signal combining gains & losses.

Notes: these features were implemented in ``engineer_features()`` and wired into a ``FunctionTransformer`` inside the pipeline.

****CV Comparison: plots & tables****

- Candidate models run via 5-fold Stratified CV: Logistic Regression, Random Forest (300 trees), Gradient Boosting.
- Scoring metrics captured: precision, recall, f1, roc_auc (per-fold).

Boxplot (per-fold distributions of Precision and ROC AUC):

![CV boxplot]([Week 4 CMIT B2/Day3/charts/boxplot.png](#))

CV summary frame structure (columns produced by the notebook):

	model	precision_mean	precision_std	recall_mean	recall_std	f1_mean	f1_std	roc_auc_mean	roc_auc_std	fit_time_sec
--	-------	----------------	---------------	-------------	------------	---------	--------	--------------	-------------	--------------

	---	---	---	---	---	---	---	---	---	
--	-----	-----	-----	-----	-----	-----	-----	-----	-----	--

	Logistic Regression	(run notebook to fill)								
--	---------------------	------------------------	--	--	--	--	--	--	--	--

	Random Forest	(run notebook to fill)								
--	---------------	------------------------	--	--	--	--	--	--	--	--

	Gradient Boosting	(run notebook to fill)								
--	-------------------	------------------------	--	--	--	--	--	--	--	--

Quick instructions to export numeric CV summary (run in the same environment used for the notebook):

```
```python
```

```
inside Week_4_CMIT_B2/Day3 notebook environment
```

```
cv_summary_df.to_csv('Week_4_CMIT_B2/Day3/charts/cv_summary.csv')
print(cv_summary_df)
```

```
'''
```

```

```

### **\*\*Statistical Test Results\*\***

Tested difference in mean Precision between the top two models (by mean Precision) using:

- Paired t-test (scipy.stats.ttest\_rel)
- Wilcoxon signed-rank test (scipy.stats.wilcoxon)

Notebook code snippet used to compute and print the results:

```
```python
precision_scores_by_model = {name: cv_results[name]['test_precision'] for name in
candidate_models}

mean_precision = {name: scores.mean() for name, scores in
precision_scores_by_model.items()}

ranked_models = sorted(mean_precision, key=mean_precision.get, reverse=True)

top1, top2 = ranked_models[0], ranked_models[1]

from scipy import stats

t_stat, p_value_t = stats.ttest_rel(precision_scores_by_model[top1],
precision_scores_by_model[top2])

w_stat, p_value_w = stats.wilcoxon(precision_scores_by_model[top1],
precision_scores_by_model[top2])

print(t_stat, p_value_t)
```

```
print(w_stat, p_value_w)
```

```
...
```

```
---
```

****Recommended Models & Features for Tuning****

Recommendations (prioritized):

- Primary tuning target: Random Forest

- Why: ensemble, typically strong off-the-shelf, handles engineered features and interactions without scaling.

- Hyperparameters to tune: `n\_estimators` (try 200, 300, 500), `max\_depth` (None, 8, 12, 20), `min\_samples\_leaf` (1, 3, 5), `max\_features` ('sqrt', 0.3, 0.5), `class\_weight` (None, 'balanced').

- Feature-focused ideas: try removing/adding `edu\_hours\_interaction`, `net\_capital`, and test `log\_capital\_gain` vs raw scaled `capital-gain`.

- Secondary tuning target: Gradient Boosting Classifier

- Why: often improves ROC AUC and precision after careful learning-rate/regularization tuning.

- Hyperparameters: `n\_estimators` (100, 300, 500), `learning\_rate` (0.01, 0.05, 0.1), `max\_depth` (3, 5, 8), `min\_samples\_leaf` (1, 3), `subsample` (0.6, 0.8, 1.0).

- Consider `HistGradientBoostingClassifier` for faster tuning on large feature sets.

- Interpretability fallback: Logistic Regression

- Why: simple, fast, interpretable coefficients; worth tuning regularization (C) and experiment with `penalty`/solver or class weights.

- Hyperparameters: `C` grid (0.01, 0.1, 1, 10), `penalty` (l2), consider feature selection (`SelectKBest`) prior to fitting for sparser model.

Feature-selection & pipeline recommendations:

- Try `SelectKBest` with `k` swept over [30, 50, 65, 100] (note notebook used `k=65` as an example).
- Test pipelines with and without the engineered categorical bins (`age_bucket`, `hours_bucket`) converted via one-hot vs. ordinal encoding.
- Evaluate `edu_hours_interaction` ablation: build models with/without the interaction to measure marginal contribution to Precision and ROC AUC.